Football Player Intelligence System

AI/ML Competition

Data & Feature Engineering Reference

Complete Guide: Collection → Cleaning → Features → Model Input

---

Document Type:	Technical Data Reference
Project Phase:	Weeks 1–2 (Data Engineering)
Primary Owner:	Data Engineer + Integration Lead
Status:	Active — in use from Day 1

This document is the single source of truth for every data decision made in this project. Every team member must read Sections 1–4 before writing any code.

Contents

1 Project Data Overview

1.1 What This System Does with Data

The Football Player Intelligence System ingests publicly available football player statistics and learns a mapping from a player's observable attributes (age, skill ratings, position) to their **market value**. Once trained, the model can:

1. **Predict** the fair market value of any player given their attributes.
2. **Identify undervalued players** — those whose predicted value significantly exceeds their listed market price.
3. **Recommend players** that fit a club's positional needs and budget constraints.

1.2 Data Flow Summary

Raw Sources	→	Cleaning	→	Feature Eng.	→	Model
FIFA 23 CSV		Remove nulls		Engineered cols		XGBoost
Transfermarkt CSV		Parse values		Log-transform		Prediction
(optional: FBref)		Merge/filter		One-hot encode		SHAP

2 Dataset Sources

2.1 Primary Dataset — FIFA 23 Complete Player Dataset (Kaggle)

Action required: Go to `https://www.kaggle.com` and search for `"FIFA 23 complete player dataset"`. Download the CSV file (typically named `players_23.csv` or similar). You need a free Kaggle account.

2.1.1 Why This Dataset?

- Contains ~18,000 players — enough for a statistically meaningful model.
- Provides 70+ features including all physical skill ratings, wages, nationality, club, and market value.
- Market value is already encoded (e.g. `€105M`, `€500K`) — you just need to parse it.
- This dataset **alone is sufficient** to build a working, competition-quality model.

2.1.2 Key Raw Columns

Column Name	Type	Description
`short_name`	string	Player display name (e.g. "L. Messi")
`long_name`	string	Full name — used for merging with Transfermarkt
`age`	int	Player age in years
`dob`	date	Date of birth
`nationality_name`	string	Country of nationality

Column Name	Type	Description
club_name	string	Current club
league_name	string	League the club plays in
overall	int	Current overall skill rating (0–99)
potential	int	Peak projected rating (0–99)
value_eur	string	Market value e.g. "€105M" — must be parsed
wage_eur	string	Weekly wage e.g. "€200K"
player_positions	string	Comma-separated positions e.g. "ST, CF"
pace	float	Speed composite rating
shooting	float	Shooting composite rating
passing	float	Passing composite rating
dribbling	float	Dribbling composite rating
defending	float	Defending composite rating
physic	float	Physicality composite rating
preferred_foot	string	"Left" or "Right"
weak_foot	int	Weak foot rating (1–5)
skill_moves	int	Skill moves rating (1–5)
international_reputation	int	Fame/reputation rating (1–5)
work_rate	string	e.g. "High/Medium" attack/defense work rate
body_type	string	Body type category
height_cm	int	Height in centimetres
weight_kg	int	Weight in kilograms

Table 1: Key columns from the FIFA 23 raw dataset

2.1.3 Critical Parsing Step — Market Value

The `value_eur` column is stored as a string with currency symbols and suffix letters. You **must** convert it to a numeric float before any analysis:

Expected output: Around 14,000–16,000 players will have valid market values after removing free agents and reserve players with no listed value.

2.2 Secondary Dataset — Transfermarkt Player Valuations (Kaggle)

Action: On Kaggle, search `"Transfermarkt player valuation dataset"`. This is a pre-scraped CSV — no scraping required. Download and place it in `data/raw/`.

2.2.1 Why Add This?

Transfermarkt is the industry standard for football player market valuations. While FIFA already includes a value field, Transfermarkt's values are:

- More frequently updated (crowd-sourced community + professional analysts).
- Used by real football clubs and journalists as a reference.
- Useful as a **second label** to validate and cross-check your FIFA value labels.

2.2.2 Merging Strategy

Merging is the hardest part of this data phase. Player names are spelled differently across sources (e.g. "Kylian Mbappé" vs "K. Mbappé" vs "Mbappe Lottin"). Never merge on ID alone.

Merge key: `player_name + nationality + approximate_age`

> **Decision Rule:** If your merge success rate is below 80%, **do not use the merged dataset**. Use FIFA alone. A clean single-source dataset beats a poorly merged dual-source dataset every time.

2.3 Optional Dataset — FBref Advanced Statistics

- **Source:** FBref.com — advanced stats like xG (expected goals), progressive carries, pressures, tackles, etc.
- **When to use:** Only if the FIFA + Transfermarkt merge completes successfully and you have spare time in Week 2.
- **Risk:** FBref covers only top-5 European leagues (∼2,500 players). Adding it means you lose ∼75% of your dataset unless you handle the missing values carefully.
- **Recommendation:** Skip unless your model's MAE on FIFA-only data is poor and you believe advanced stats are the missing signal.

3 Data Cleaning Protocol

3.1 Step-by-Step Cleaning Pipeline

Run these steps **in order**. Each step must be logged to the shared notebook.

3.1.1 Step 1 — Initial Inspection

3.1.2 Step 2 — Drop High-Missingness Columns

Any column missing more than 40% of its values provides insufficient signal and introduces noise. Drop them.

3.1.3 Step 3 — Impute Remaining Numeric Nulls

3.1.4 Step 4 — Parse Market Value and Filter

3.1.5 Step 5 — Remove Extreme Outliers

A handful of players (top global superstars) have values that can distort model training. We handle this using log-transformation (Section 4), but also clip the extreme top.

3.1.6 Step 6 — Save Clean Dataset

Checkpoint — End of Days 4–7: You should have a clean CSV with **12,000–16,000 players** and no critical missing values. Log the exact row and column count to the team's shared notes.

4 Feature Engineering

Feature engineering is the team's **competitive advantage**. Raw FIFA attributes alone predict value reasonably, but these engineered features capture deeper signals that raw ratings miss.

4.1 Target Variable — Log Market Value

4.1.1 Why Not Raw Euros?

Market values in football follow a **power law distribution**. A few elite players (Mbappé, Haaland) command values of 150M+, while the median professional player is worth 1–2M. Training a regression model on raw euro values causes the model to:

- Obsess over getting the top 1% right (high absolute error there dominates loss).
- Perform poorly on the 99% of "average" players.
- Be numerically unstable due to the enormous value range.

4.1.2 The Fix — Log Transformation

Apply the natural logarithm to compress the range:

$$y = \log(1 + \text{market\_value\_eur}) \tag{1}$$

This is implemented as `np.log1p()` (the `+1` prevents `log(0)` errors). At prediction time, always invert with:

$$\hat{\text{market\_value}} = e^{\hat{y}} - 1 = \texttt{np.expm1}(\hat{y}) \tag{2}$$

4.2 Engineered Features

4.2.1 Feature 1 — Age–Potential Gap

$$\texttt{age\_potential\_gap} = \texttt{potential} - \texttt{overall} \tag{3}$$

What it captures: The room for growth remaining in a player. A 20-year-old with an overall of 72 and potential of 92 has a gap of 20 — this is a classic undervalued signal because their current market price reflects their current ability (72), but their future value will be far higher (92).

Why it matters for the model: Neither `potential` nor `overall` alone captures this. The gap is a distinct signal — young players with large gaps are systematically underpriced.

4.2.2 Feature 2 — Value-per-Rating Ratio

$$\texttt{value\_per\_rating} = \frac{\texttt{market\_value\_eur}}{\texttt{overall} + 1} \tag{4}$$

What it captures: How much the market is paying per unit of ability. High `value_per_rating` = overpriced (market pays premium beyond raw skill). Low `value_per_rating` = potentially underpriced.

Use in demo: Sorting by `value_per_rating` ascending gives your "undervalued players" list — predicted value ≫ actual price.

4.2.3 Feature 3 — Position Group (One-Hot Encoded)

Raw FIFA positions are too granular (there are 27+ positions: LW, RW, CAM, ST, etc.). Group them into 4 meaningful categories:

Position Group	Includes
GK	GK
DEF	CB, LB, RB, LWB, RWB
MID	CDM, CM, CAM, LM, RM
ATT	LW, RW, ST, CF, LF, RF

One-hot encode these groups — **never use label encoding for nominal categories**. Label encoding (0, 1, 2, 3) implies an ordinal relationship (DEF > MID makes no sense).

4.3 Final Feature Set for the Model

Table 2 defines every column in the `model_ready.csv` file — the locked output of the data pipeline and the input contract for the ML Engineer.

#	Feature Name	Type	Range / Notes
— *Raw Numeric Features* —			
1	`age`	int	15–45; younger = higher growth potential
2	`overall`	int	40–99; strongest single predictor of value
3	`potential`	int	40–99; future ceiling rating
4	`pace`	float	0–99; sprint/acceleration composite
5	`shooting`	float	0–99; finishing, long shots, volleys
6	`passing`	float	0–99; vision, crossing, short pass
7	`dribbling`	float	0–99; ball control, agility, balance
8	`defending`	float	0–99; tackles, interceptions, marking
9	`physic`	float	0–99; strength, stamina, aggression
— *Engineered Features* —			
10	`age_potential_gap`	int	potential overall; 0–40; growth room signal
11	`value_per_rating`	float	market_value / (overall+1); price-per-skill
— *One-Hot Encoded Position* —			
12	`position_group_GK`	binary	1 = goalkeeper, 0 = otherwise
13	`position_group_DEF`	binary	1 = defender, 0 = otherwise

#	Feature Name	Type	Range / Notes
14	`position_group_MID`	binary	1 = midfielder, 0 = otherwise
15	`position_group_ATT`	binary	1 = attacker, 0 = otherwise
— *Target Variable* —			
16	`log_value`	float	log(1 + market_value_eur); model trains on this
— *Metadata (not model inputs)* —			
17	`short_name`	string	Player display name; for demo/UI use only
18	`market_value_eur`	float	Raw euro value; for evaluation and display

Table 2: Complete feature set — `data/features/model_ready.csv`

5 Exploratory Data Analysis (EDA)

EDA is **mandatory**. Every team member must review the notebook output before the dataset is locked. The following analyses must be completed and saved.

5.1 Required EDA Outputs

1. **Market Value Distribution** — histogram of raw values and log values side by side. Confirms the log-transform is working.
2. **Value by Position Group** — box plot of market value per position. Expected: attackers have the highest median and variance.
3. **Correlation Heatmap** — correlation matrix of all numeric features vs. `log_value`. Expected: `overall` will have the highest correlation ($r > 0.7$).
4. **Age vs Value Scatter** — coloured by position group. Should show the classic inverted-U curve (value peaks at age 25–28 for most positions).
5. **Undervalued Player Preview** — list the 20 players with the largest gap between `value_per_rating` (low) and `potential` (high). These are your demo stars.

6 Model Inputs and Outputs — Formal Specification

This section is the **data contract** between the Data Engineer and the ML Engineer. The ML Engineer must not deviate from this spec without team agreement.

6.1 Input Tensor Specification

For any player, the model receives a vector $\mathbf{x} \in \mathbb{R}^{14}$:

$$\mathbf{x} = \begin{bmatrix} \texttt{age} & \texttt{overall} & \texttt{potential} & \texttt{age\_potential\_gap} \\ \texttt{pace} & \texttt{shooting} & \texttt{passing} & \texttt{dribbling} \\ \texttt{defending} & \texttt{physic} & \texttt{pos\_GK} & \texttt{pos\_DEF} \\ \texttt{pos\_MID} & \texttt{pos\_ATT} & & \end{bmatrix}^T \tag{5}$$

All values must be numeric. No strings, no NaNs, no unnormalised categories.

6.2 Output Specification

$$\hat{y} = f(\mathbf{x}) \quad \text{where } \hat{y} \approx \log(1 + \text{market\_value\_eur}) \tag{6}$$

$$\widehat{\text{market\_value\_eur}} = e^{\hat{y}} - 1 = \texttt{np.expm1}(\hat{y}) \tag{7}$$

The final API endpoint returns `predicted_value_eur` (float) and `predicted_value_m` (float, millions) — both derived from `np.expm1()`.

7 File Naming and Directory Conventions

All team members must follow these conventions without exception:

Path	Contents
`data/raw/`	Original downloaded files. **Never edit these.**
`data/processed/players_clean.csv`	Output of cleaning pipeline.
`data/features/model_ready.csv`	Locked dataset with all features.
`notebooks/01_eda.ipynb`	EDA notebook — shared and reviewed.
`notebooks/02_modelling.ipynb`	Model experiments — ML Engineer only.
`models/xgboost_player_value.pkl`	Trained model file, committed to repo.

Never commit raw data files to GitHub. Add `data/raw/` to your `.gitignore`. Raw Kaggle CSVs can be several hundred MB and will bloat the repository. Instead, document the Kaggle dataset URL in the README so any team member can re-download it.

8 Common Errors and Fixes

Error	Cause	Fix
`ValueError: could not convert string to float`	`value_eur` not parsed yet	Run `parse_value()` before any numeric operations
`KeyError: 'position_group_ATT'`	Player positions contained an unmapped value (e.g., "SUB")	Print unmapped values; add to `position_map` or drop those rows
`NaN` in model input	Imputation step skipped or a new column added after imputation	Re-run the full cleaning pipeline from top
Merge rate below 80%	Name spellings too different between FIFA and Transfermarkt	Lower the fuzzy threshold to 80 (from 85) OR use FIFA data only
Model predicts negative values	Log-scale output not converted back	Always wrap output in `np.expm1()`

Error	Cause	Fix
`position_group_` columns missing after reload	`get_dummies` not saved correctly	Save AFTER encoding; reload and verify column names
`MemoryError` on large merge	Fuzzy matching is $O(n^2)$	Sample 5,000 rows for testing; run full merge overnight

Table 3: Common data errors and their resolutions

9 Week 2 Completion Checklist

Before declaring the data phase complete and locking the dataset, every item below must be checked off:

- ☐ `data/raw/fifa23_players.csv` downloaded and placed in repo.
- ☐ `data/processed/players_clean.csv` generated with ≥12,000 rows.
- ☐ All 5 EDA plots generated and saved to `notebooks/`.
- ☐ `data/features/model_ready.csv` generated with exactly 18 columns (14 features + `log_value` + 3 meta).
- ☐ Zero null values in `model_ready.csv` (verified with `assert`).
- ☐ Dataset row and column count logged to team shared notes.
- ☐ `data/raw/` added to `.gitignore`.
- ☐ ML Engineer has confirmed they can load `model_ready.csv` and run a 5-line linear regression without errors.
- ☐ **Dataset locked** — team agrees no further schema changes.

If all boxes are checked, the data phase is complete. The ML Engineer takes over in Week 3. The Integration Lead moves to model evaluation support.